

Question Bank — Week 4

Dynamic arrays and amortised analysis (Lecture 04) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Questions: *week04-questions.md*. Commit to your own answer before reading one here.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M07	c	M13	a	M19	c
M02	c	M08	d	M14	b	M20	d
M03	b	M09	a	M15	c	M21	a
M04	d	M10	b	M16	d	M22	b
M05	a	M11	c	M17	a		
M06	b	M12	d	M18	b		

W4-M01 — a. len is the number of elements the user stored. The capacity (b) is internal; showing it would expose the implementation.

W4-M02 — c. Capacity $1 \rightarrow 2$ (2nd append) $\rightarrow 4$ (3rd) $\rightarrow 8$ (5th).

W4-M03 — b. Resizes happen when the size reaches 1, 2, 4, ..., 512 — ten powers of two below 1,000 — so exactly 10 ($2^{10} = 1024$). 999 (d) is growing by one.

W4-M04 — d. Every append copies everything already there: $0 + 1 + \dots + (n-1) = n(n-1)/2$.

W4-M05 — a. Copies are 100, 200, 300, ...: about $n^2/200$ in total — still quadratic, so $O(n)$ per append on average. A constant step only changes the constant. (d) confuses the step size with the cost.

W4-M06 — b. Fewer than $3n$ units of work for n appends: under 3 each.

W4-M07 — c. The append that finds the array full copies all n elements. The amortised bound does not change that single worst case.

W4-M08 — d. Amortised analysis bounds the total over **any** sequence — a guarantee. Average case (c) averages over an assumed distribution of inputs.

W4-M09 — a. $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2n$, since $2^k < n$.

W4-M10 — b. Appends that find the array full are those made when the size is a power of two: the 2nd, 3rd, 5th, 9th, 17th, 33rd, ... (one more than a power of two).

W4-M11 — c. A gap between the grow threshold (full) and the shrink threshold (one quarter) means many cheap operations must happen between resizes. Halving at one half (a) leaves no gap.

W4-M12 — d. For example: full at capacity 8, append (grow to 16), pop (shrink to 8), append (grow), ... — an $O(n)$ copy on every operation.

W4-M13 — a. Capacities 4, 8, 16, 24, 32, 40, 52, 64, 76, ... on Python 3.13.

W4-M14 — b. `sys.getsizeof` returns the header plus one reference (8 bytes) per slot of **capacity**; subtract the empty list's size and divide by 8.

W4-M15 — c. $-1 + \text{size} = 4$: the last *used* slot, not the last slot (a).

W4-M16 — d. All n elements shift right — whether or not a resize is needed.

W4-M17 — a. Nothing moves; clear the slot and decrement the size.

W4-M18 — b. The copies at each resize double, so the last resize copies more than all the earlier ones together, and it copies fewer than n .

W4-M19 — c. Any factor $r > 1$ gives a geometric series: about $n \cdot \frac{r}{r-1}$ copies — $3n$ for $r = 1.5$ against $2n$ for $r = 2$ — while the unused space after a resize is about a third rather than a half.

W4-M20 — d. Growing from c to $2c$ with $c + 1$ elements leaves $c - 1$ slots unused: about half.

W4-M21 — a. Stack keeps its top at the end of a `DynamicArray`. `CircularQueue` and the open-addressing map use a plain `Array`; the tree uses nodes.

W4-M22 — b. One coin for the append's own write, two saved to pay for copying at the next doubling.

Part B — Short answer and essay

W4-E1 (5)

- **Definition:** the amortised cost of an operation is the maximum total cost of any sequence of n operations, divided by n . It is a guarantee over sequences, not an average over inputs.
- **Writes:** each of the n appends writes one element: n .
- **Copies:** resizes happen when the size reaches $1, 2, 4, \dots, 2^k$ with $2^k < n$, each copying that many elements.
- **Sum:** $1 + 2 + \dots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k < 2n$.
- **Conclusion:** total $< 3n = O(n)$, so the amortised cost of one append is $< 3 = O(1)$. (The accounting argument — 3 coins per append — is equally acceptable.)

W4-E2 (4)

- Each resize copies **everything** stored so far.

- **Constant step c:** resizes come every c appends, at sizes about c, 2c, 3c, ...; the amounts copied grow **linearly**, and a sum of n/c linearly growing terms is about $n^2/(2c) = \Theta(n^2)$. A larger c only shrinks the constant.
- **Factor r:** resizes come at sizes growing **geometrically**; each one copies r times more than the last, so the whole sum is dominated by its last term, which is less than n: total $\Theta(n)$.
- The difference is linear against geometric spacing of the resizes — not the size of the step.

W4-E3 (3)

- **Worst case $O(n)$:** a single append that finds the array full must copy all n elements before writing.
- **Amortised $O(1)$:** such appends are exponentially rare; any n appends cost under 3n in total. Both are true because they measure different things — one operation, against a whole sequence.
- **When the worst case matters:** when one slow operation is itself a failure — real-time systems, a game frame, a latency-critical request. Those reserve capacity in advance.

W4-E4 (4)

- **Thrashing:** operations alternating around a single boundary force a resize almost every time, so each costs $O(n)$.
- **Example** (grow at full, shrink at half full): capacity 8, size 8. append → grow to 16 (copy 8); pop → size 8 = half of 16 → shrink to 8 (copy 8); append → grow (copy 8); and so on.
- **Rule:** grow when full, but shrink (halve) only when the size falls to **one quarter** of the capacity.
- **Why it works:** after any resize the array is about half full, so at least about n/4 cheap operations must happen before the next resize, which pays for it: amortised $O(1)$ for any mix of appends and pops.

W4-E5 (3)

- A **larger factor** resizes less often (fewer total copies, e.g. under $2n$ for $\times 2$) but can leave more capacity unused (up to about half).
- A **smaller factor** wastes less memory but copies more (about $n r / (r - 1)$ in total — around $9n$ for $r = 1.125$).
- Any factor > 1 keeps append amortised $O(1)$. CPython chooses a small one because Python programs create huge numbers of lists, most of them small, so the saved memory matters more than the extra copying.

Part C — Trace the growth

W4-T1. Resizes on appends 2, 3, 5 and 9 (sizes 1, 2, 4, 8 were full). Capacity **16**, **4** resizes, **15** elements copied ($1 + 2 + 4 + 8$).

W4-T2. Resizes on appends **2, 3, 5, 9, 17**. Copies $1 + 2 + 4 + 8 + 16 = 31$; writes 20; total **51**; average $\$51 / 20 = \$ 2.55$ per append — under 3, as the proof promises.

W4-T3. Capacities $1 \rightarrow 4 \rightarrow 7 \rightarrow 10 \rightarrow 13$. Resizes on appends **2, 5, 8, 11**. Final capacity **13**; copies $\$1 + 4 + 7 + 10 = \22 . The copies grow linearly — the signature of a constant step.

W4-T4.

Operation	Size	Capacity	Why
append(1)	1	1	room
append(2)	2	2	full $\rightarrow$ double
append(3)	3	4	full $\rightarrow$ double
append(4)	4	4	room
append(5)	5	8	full $\rightarrow$ double
pop() $\rightarrow$ 5	4	8	$4 > 8 // 4 = 2$
pop() $\rightarrow$ 4	3	8	$3 > 2$
pop() $\rightarrow$ 3	2	4	$2 \leq 2 \rightarrow$ halve
pop() $\rightarrow$ 2	1	2	$1 \leq 4 // 4 = 1 \rightarrow$ halve

W4-T5. Appends **2, 3, 5, 9, 17, 33**: each is one more than a power of two, because the array is full exactly when its size is a power of two. The expensive appends double their spacing each time — which is why their cost averages out.

Part D — Proofs

W4-P1. Resizes happen when the size reaches $1, 4, 16, \dots, 4^k$ with $4^k < n$, and each copies that many elements. The copies total

$$1 + 4 + \dots + 4^k = \frac{4^{k+1} - 1}{3} < \frac{4}{3} \cdot 4^k < \frac{4n}{3}.$$

Adding the n writes, the total is below $n + \frac{4n}{3} = \frac{7n}{3} = O(n)$, so append is amortised $O(1)$. ■
(The same proof works for any factor $r > 1$: copies $< n \cdot \frac{r}{r-1}$.)

W4-P2. Capacities are $1, 1 + c, 1 + 2c, \dots$. A resize happens when the size equals the capacity, $1 + jc$, for every j with $1 + jc < n$; call the number of resizes m . Then $m \geq (n - 1)/c$, and resize j copies $1 + jc$ elements, so

$$\text{copies} = \sum_{j=0}^{m-1} (1 + jc) = m + c \frac{m(m-1)}{2} \geq \frac{n-1}{c} + \frac{(n-1)(n-1-c)}{2c} = \frac{(n-1)(n+1-c)}{2c}.$$

Expanding, $\frac{(n-1)(n+1-c)}{2c} = \frac{n^2}{2c} - \frac{n}{2} + \frac{c-1}{2c} \geq \frac{n^2}{2c} - n$. For fixed c this is $\Omega(n^2)$: no constant step can make append amortised $O(1)$. ■

Part E — Find and fix the bug

W4-B1. The test should be `==`: with `>`, the array is never grown when it is exactly full, so the write goes one past the end of the Array — the second append raises `IndexError: Array index 1 out of range for length 1`. **Fix:** `if self._size == self._capacity:`.

W4-B2. The bound is the **capacity**, so reading an unused slot returns `None` instead of raising: with `[1, 2, 3]` stored (capacity 4), `a[3]` gives `None`. `test_index_out_of_range_raises` fails. **Fix:** `0 <= index < self._size`.

W4-B3. It grows by a **constant** (growth is 2, so `+ 2`), not by a factor: 1,000 appends resize about 500 times, and append is amortised $O(n)$. **Fix:** `self._resize(self._capacity * self.growth)`.

W4-B4. After shifting, the slot at the old last position still refers to the last element (and, for a pop from the end, to the removed value). The array keeps that object alive, so the garbage collector cannot reclaim it — a memory leak for large objects. **Fix:** after `self._size -= 1`, add `self._block[self._size] = None`.

Part F — Write the code

W4-C1

```
def capacity_after(n, factor=2):
    capacity, size = 1, 0
    for _ in range(n):
        if size == capacity:
            capacity *= factor
        size += 1
    return capacity
```

W4-C2

```
def copies_for(n, grow):
    capacity, size, copies = 1, 0, 0
    for _ in range(n):
        if size == capacity:
            copies += size          # the resize copies everything stored
            capacity = grow(capacity)
        size += 1
    return copies
```

`copies_for(100_000, lambda c: 2 * c)` is 131,071 and `copies_for(100_000, lambda c: c + 1)` is 4,999,950,000 — Lecture 04's numbers.

W4-C3

```

class ShrinkingArray:
    def __init__(self):
        self._block = Array(1)
        self._size = 0
        self.resize_count = 0

    @property
    def capacity(self):
        return len(self._block)

    def _resize(self, capacity):
        block = Array(capacity)
        for i in range(self._size):
            block[i] = self._block[i]
        self._block = block
        self.resize_count += 1

    def append(self, value):
        if self._size == self.capacity:
            self._resize(2 * self.capacity)
        self._block[self._size] = value
        self._size += 1

    def pop(self):
        if self._size == 0:
            raise IndexError("pop from empty array")
        self._size -= 1
        value = self._block[self._size]
        self._block[self._size] = None          # Let the object be collected
        if self._size <= self.capacity // 4 and self.capacity > 1:
            self._resize(self.capacity // 2)
        return value

    def __len__(self):
        return self._size

    def __getitem__(self, index):
        if not 0 <= index < self._size:
            raise IndexError(index)
        return self._block[index]

```

The thrashing test: it fills the array to size 8, capacity 8, then alternates append and pop. The first append grows to 16 (size 9); the pop leaves size 8, which is above $16 // 4 = 4$, so nothing happens — and the same for every later pair: one resize in total. With a half rule, size $8 \leq 16 // 2$ would shrink back to 8, and the next append would grow again: a resize on every operation, and

the test fails.